Strategic and Technical Validation of the Daanan Proposal for the AppBuilders 2026 Hackathon

Executive Overview of the Daanan Initiative

The Daanan project proposal outlines a highly ambitious mobile application designed to provide local, offline-first artificial intelligence commute directions tailored to the unique, informal transit ecosystems of the Philippines. Proposed for the AppBuildersPH Hackathon scheduled for October 9–10, 2026, the application intends to leverage on-device large language models (LLMs) to parse natural language queries in a localized Taglish dialect and retrieve transit routes via a local vector database[1]. The core premise rests on the digitization of tacit, community-held transit knowledge—such as jeepney signboards, tricycle fares, and unofficial drop-off landmarks—which are currently absent from standard General Transit Feed Specification (GTFS) datasets and conventional routing applications like Google Maps[1].

The comprehensive evaluation of this proposal requires a rigorous assessment against the specific parameters of the AppBuilders hackathon, which mandates that projects demonstrate "Local AI" capabilities where meaningful computation occurs natively on the user's device rather than relying on cloud infrastructure[2]. The following analysis scrutinizes the strategic viability, technical architecture, and overall execution feasibility of the Daanan proposal. The assessment explores the integration of edge-optimized language models, mobile-native vector search environments, and the immense memory and computational demands of offline geospatial routing engines. By isolating the architectural vulnerabilities and strategic strengths of the proposal, this report provides a definitive roadmap for optimizing the Daanan project for maximum impact within a constrained 24-to-48-hour hackathon environment.

Alignment with Hackathon Evaluation Criteria and Strategic Constraints

The AppBuildersPH 2026 evaluation rubric is strictly weighted to assess both the practical utility of the application and the authenticity of its local AI implementation, demanding a working product built substantially during the hackathon period[2]. The Daanan proposal must be validated across five distinct judging pillars to ascertain its competitive viability, while simultaneously aligning its strategic timeline with the reality of an overnight sprint.

Problem Validation and Utility

Accounting for twenty-five percent of the total score, the problem space must present a genuine friction point for a clearly defined user demographic[2]. The proposal astutely identifies the informal transit sector in the Philippines as its primary domain, recognizing a profound data deficit in existing platforms. Official data from the Land Transportation Franchising and Regulatory Board (LTFRB) reveals immense fragmentation, with over 1,767 unconsolidated

jeepney routes nationwide, many of which operate on highly localized, short-distance corridors that defy traditional mapping and algorithmic routing[4]. Existing platforms like Sakay.ph provide robust coverage within the National Capital Region (NCR) utilizing historic GTFS feeds derived from initiatives like the Philippine Transit App Challenge, but exhibit significant blind spots in peripheral provinces and regarding informal modes like tricycles and unfranchised utility vehicles[1].

By employing a rigorous consulting framework—specifically the concept of mutually exclusive and collectively exhaustive (MECE) categorization alongside the "5 Whys" root cause analysis—the proposal correctly identifies the fundamental bottleneck[1]. The knowledge required to navigate informal transit is tacit, unstructured, embedded in community forums like Reddit's r/HowToGetTherePH, and highly vulnerable to internet connectivity dead zones[1]. The target demographic consists of provincial and inter-city commuters who frequently encounter weak mobile data signals or must aggressively ration prepaid data allocations[1]. Daanan's utility is immediately apparent; by decoupling transit knowledge from cloud dependency, it directly solves a high-friction daily challenge for millions of commuters, firmly satisfying the first quartile of the hackathon's grading rubric.

Local AI Implementation Efficacy

An additional twenty-five percent of the evaluation hinges on whether local inference provides a meaningful advantage over cloud-based alternatives[2]. The rules explicitly require that the application must remain genuinely useful when the cloud disappears[2]. Daanan's architecture dictates that natural language understanding, geographic entity resolution, and route synthesis all occur entirely offline[1]. This is not merely a technical novelty but a functional necessity dictated by the use case. A commuter stranded in a mobile data dead zone along Marcos Highway cannot query a cloud-based OpenAI or Anthropic model to determine which jeepney signboard indicates their destination. By bundling region-specific SQLite databases with quantized edge models, the application guarantees continuous operability regardless of network conditions[1]. This represents the exact paradigm shift the hackathon organizers intend to reward, contrasting sharply with thin-wrapper applications that merely pass API calls to cloud endpoints[2].

Technical Execution and Reliability

Technical execution, weighted at twenty percent, will dictate whether the prototype functions reliably during the mandatory live demonstration[2]. The proposal outlines a sophisticated pipeline: a localized natural language query is processed by a small language model (SLM) to extract intents, which then interfaces with a geospatial vector database to retrieve relevant route legs, followed by algorithmic routing via an offline engine, and concluding with natural language generation for the final user response[1]. While conceptually sound, this pipeline introduces immense memory and processing constraints on consumer-grade mobile hardware[7]. The validation of this architecture requires a granular examination of device Random Access Memory (RAM) limitations, quantization strategies, and mobile processor architectures, which necessitates strict architectural compromises that the current proposal does not yet fully acknowledge.

Innovation and Product Presentation

The final thirty percent of the evaluation is divided equally between innovation and presentation quality[2]. The innovation lies in the novel synthesis of disparate technologies. Combining local Retrieval-Augmented Generation (RAG) with offline geographic information systems (GIS) to map unstructured, informal human knowledge is a highly defensible unique insight[1].

Furthermore, the proposal demonstrates a clear understanding of optimal pitch mechanics, adapting a Y Combinator seed deck format into a 99-second, seven-slide presentation that prioritizes traction, root-cause validation, and a seamless live demonstration over technical jargon[1].

However, a critical vulnerability exists in the proposed timeline. The document describes a "6-week prototype" leading to a go/no-go decision for a pilot program[1]. The AppBuilders participant briefing strictly enforces a rapid, remote build cycle, with code freezes and final GitHub repository submissions due by 10:00 AM on October 10[2]. To survive the hackathon, the MVP scope must be violently compressed. The team must abandon all attempts to build a backend extraction pipeline during the event and instead manually compile a highly constrained dataset for the demonstration corridor.

Spec-Driven Development and Validation Engine

The proposal highlights a reliance on "spec-driven development," where comprehensive documentation precedes the deployment of AI coding agents like Devin, Cursor, or Claude Code[1]. The hackathon rules explicitly permit AI-assisted development and the use of tools like Devin, provided they are disclosed[2].

Preventing Context Rot in Agent Harnesses

The strategy of producing a Product Requirements Document (PRD), Software Design Document (SDD), Entity Relationship Diagram (ERD), and sitemap before writing code is a highly mature approach to managing autonomous or semi-autonomous AI agents[1]. The proposal correctly identifies the dual threats of "context rot" and "context poisoning" that plague long-running AI coding sessions[1]. Context rot occurs as the LLM's output degrades over extended conversations due to the dilution of initial instructions, a phenomenon documented in comprehensive model testing[1]. Context poisoning, highlighted by deep reinforcement learning failures, occurs when an agent internalizes a hallucination and continuously references it in subsequent iterations[1].

By treating the offline specification documents as the immutable source of truth and utilizing fresh, disposable chat sessions for feature implementation, the development team can maintain high-velocity output throughout the overnight sprint[1]. Maintaining a progress log and committing frequently allows these agent harnesses to pick up cleanly without succumbing to "context clash"[1]. This methodology is practically mandatory for a small team attempting to construct a localized LLM application, a vector database, and a routing engine within 48 hours.

The Validation Loop

The proposal's reliance on The Mom Test methodologies for continuous validation—focusing on actual user behavior and monetary signals rather than hypothetical future usage—is

conceptually sound[1]. However, conducting 5 to 10 user interviews and iterating based on survey results during a 24-hour hackathon is logistically improbable[1]. The validation engine must be front-loaded. The team should execute these interviews prior to the event kickoff to establish their traction metrics for the "Opportunity" and "Problem at scale" slides[1]. During the hackathon, validation should be strictly limited to usability testing of the final prototype to ensure the natural language query interface functions intuitively[1].

Architectural Feasibility: Edge Language Models

The success of Daanan relies absolutely on the capacity of a mid-range Android device to run a language model capable of parsing Taglish and formatting structured route data without succumbing to hallucinations or excessive latency[1]. The proposal lists Qwen and Gemma models, specifically targeting the 1.5 billion to 4 billion parameter range[1].

Small Language Model (SLM) Benchmarking and Selection

Selecting the appropriate foundation model requires balancing parameter count against context window capabilities, reasoning scores, and multilingual proficiency. An analysis of current SLMs reveals distinct architectural trade-offs that dictate their suitability for local RAG applications:

Model Family	Parameter Count	MMLU Score	GSM8K (Math)	HumanEval (Code)	Context Window	Relative Speed
Qwen 2.5 3B	3.0 Billion	~65%	~79%	~65%	128,000 Tokens	Fast
Llama 3.2 3B	3.2 Billion	~63%	~77%	~45%	128,000 Tokens	Fast
Phi-3.5 Mini	3.8 Billion	~69%	~86%	~62%	128,000 Tokens	Moderate
Gemma 2 2B	2.0 Billion	~52%	~40%	~20%	8,000 Tokens	Fastest

Data aggregated from comparative small model showdowns and baseline capability testing[8].
Gemma 2 2B: While Google's Gemma 2 2B model is the most compact option and boasts the highest theoretical throughput (generating 800-1000 tokens per second on a single desktop GPU in FP16), it suffers from severe limitations that disqualify it for this specific architecture[8]. It exhibits a strict 8K context window limitation[8]. While 8K is generally sufficient for a brief transit query, RAG architectures require the injection of substantial background data—in this case, multiple geographic route legs, fare structures, and system prompts dictating output formatting.

An 8K limit restricts the amount of vector data that can be provided simultaneously. Furthermore, Gemma 2 2B scores significantly lower on reasoning and math benchmarks, dropping to ~40% on GSM8K and ~20% on HumanEval, suggesting it will struggle to reliably synthesize complex, multi-leg transit instructions without hallucination[8].

Llama 3.2 3B: Meta's offering is a highly capable reasoning engine with a generous 128K context window, heavily optimized for specific architectures including Qualcomm Snapdragon devices[8]. However, its multilingual capabilities are largely limited to strong English, which presents a significant barrier when processing highly colloquial Taglish queries, unofficial landmark names, and regional slang[10]. Additionally, in standard FP16 format, it demands approximately 6GB of storage and extensive RAM during inference, routinely exceeding the typical capabilities of mid-range smartphones[7]. While aggressive post-training quantization (PTQ) to 4-bit weights and 16-bit activations (w4a16) can reduce this footprint to a manageable level, the linguistic barrier remains a liability for the Philippine market[7].

Qwen 1.5 and 2.5 Families: Developed by Alibaba, the Qwen architecture demonstrates exceptional performance in multilingual contexts, natively supporting over 100 languages with robust tokenization strategies that handle Asian dialects and mixed-language inputs efficiently[9]. The Qwen 2.5 3B variant possesses a massive 128K context window, highly advantageous for injecting multiple retrieved route legs into the prompt[8]. Furthermore, Qwen models exhibit strong instruction-following and coding capabilities (scoring ~65% on HumanEval compared to Llama's ~45%), which translates directly into the model's ability to strictly adhere to JSON or Markdown formatting templates when generating transit directions, rather than inventing fictional routes[8]. On highly optimized edge configurations, Qwen 2.5 1.5B has been benchmarked at impressive speeds, holding up remarkably well under edge workloads[13].

Given the strict requirement to process Taglish, ingest RAG context, and output structured data without geographic hallucination, a highly quantized Qwen model (specifically the 1.5B or 3B variant) emerges as the strategically superior choice for the Daanan prototype[8].

Inference Engine Mechanics: The Mobile GPU Bottleneck

The proposal specifies the use of llama.cpp, a pure C/C++ implementation that operates without external dependencies and utilizes the GGUF file format with aggressive 2-bit to 8-bit quantization capabilities[15]. While this is a highly popular framework, its deployment on Android introduces severe hardware bottlenecks that the development team must circumvent.

A critical, counter-intuitive reality in mobile machine learning deployment is that Graphics Processing Unit (GPU) offloading is not always beneficial, and can often be detrimental. Extensive benchmarking of llama.cpp on modern Android processors, specifically Qualcomm Snapdragon chipsets equipped with Adreno GPUs, reveals that OpenCL implementations are frequently broken for LLM inference[16]. The underlying issue stems from the massive memory bandwidth overhead inherent in Unified Memory Architectures (UMA) on mobile Systems-on-a-Chip (SoCs). In practical testing, passing the matrix multiplication operations back and forth between the CPU and the mobile GPU via OpenCL consumes the vast majority of processing time, resulting in abysmal performance metrics ranging from 0.5 to 5 tokens per second[16].

This bottleneck is not entirely solved by switching graphics APIs. While Vulkan backends demonstrate excellent performance improvements on desktop unified memory architectures—such as Apple Silicon or Intel Panther Lake processors, where Vulkan can deliver a 6x speedup over CPU for prompt processing—the performance on standard Android mobile SoCs often remains inferior to raw CPU execution[16]. Benchmarks indicate that Vulkan on an Adreno GPU still suffers from severe overhead, rendering it 5 to 10 times slower than optimal CPU execution[16].

Conversely, relying entirely on the CPU utilizing specialized ARM matrix instructions (I8MM) yields substantially better performance[16]. When llama.cpp detects and utilizes these CPU capabilities, inference speeds consistently reach 5 to 15 tokens per second on mid-range hardware, which is entirely viable for a localized chat application[10]. Therefore, to ensure a responsive user experience during the live hackathon demonstration, the Daanan engineering team must explicitly configure llama.cpp to prioritize CPU threads and aggressively disable OpenCL or Vulkan GPU offloading on their Android test devices[16].

Memory Management and Android OOM Constraints

The memory footprint of an LLM running via llama.cpp is calculated as the total sum of the GGUF quantized weights plus the Key-Value (KV) cache required to process the context window[19]. A 3B parameter model quantized to 4-bit precision (Q4_K_M) requires approximately 1.8GB to 2.2GB of physical RAM merely to load the model weights[7]. The KV cache expands dynamically based on the length of the prompt and the requested generation tokens[19].

On a mid-range Android phone with 6GB of total system RAM, the operating system's memory management is highly aggressive; it will instantly terminate background processes or high-memory foreground applications to protect system stability, throwing fatal Out-Of-Memory (OOM) errors[7]. To prevent the Daanan application from being killed by the Android OOM killer during a query, developers must employ rigorous memory boundaries:

1. **Strict KV Cache Limits:** Utilize the -c flag to restrict the context size to the absolute minimum required for the routing task (e.g., 2048 or 4096 tokens), rather than attempting to leverage the full 128K capability of the Qwen model[19].
2. **Output Bounds:** Set a strict maximum limit on output generation using the -n flag on every call to prevent runaway generation from exhausting memory[19].
3. **Headroom Allocation:** Utilize the --fit-target command line argument to reserve safety headroom in megabytes, ensuring llama.cpp does not attempt to consume every available byte of RAM, which guarantees a system-level crash when the Android compositor or background services demand memory[19].

The LiteRT-LM Alternative

Given the complexities of compiling llama.cpp natively for Android under time pressure, the team should heavily consider pivoting to Google's LiteRT-LM framework (the successor to TensorFlow Lite for LLMs)[22]. LiteRT-LM is a production-ready orchestration layer specifically engineered for deploying LLMs on edge devices, offering deep optimizations for Android hardware and native support for both Qwen and Gemma architectures[24].

Integrating this via the flutter_gemma ecosystem provides a highly abstracted, modular package

that supports models via .litertlm engines while handling the lower-level memory management and NPU acceleration automatically[25]. flutter_gemma natively supports Qwen 2.5 and provides pre-built modules for exactly what Daanan requires, allowing the team to focus on application logic rather than debugging C++ memory allocation on mobile hardware[26].

Data Storage and On-Device Vector Search

The proposal dictates the use of sqlite-vec for storing and retrieving route legs via vector similarity search, bypassing the need for cloud-based embedding or database APIs[1]. This is an exceptionally sophisticated and highly appropriate architectural choice for edge environments.

The sqlite-vec Paradigm

sqlite-vec is a successor to sqlite-vss, designed as an extremely lightweight, dependency-free C extension for SQLite that introduces a vec0 virtual table optimized for storing and querying float, int8, and binary vectors[6]. Because it is written in pure C without external dependencies, it operates natively wherever SQLite runs, making it universally compatible with Android and iOS devices without requiring a standalone vector database daemon or complex client-server architecture[6].

For the Daanan application, this means the entire regional route database can be shipped as a single, highly compressed pre-compiled SQLite file within the application bundle, or downloaded as a modular region-specific pack (e.g., Region IV-A)[1]. The workflow operates seamlessly: when a user enters a query such as "Pano pumunta sa SM Masinag galing Cubao," the application embeds this text into a query vector using a small, specialized on-device embedding model (which can also be managed by flutter_gemma or llama.cpp)[1]. The application then executes a native SQL MATCH query against the vec0 table to perform a K-Nearest Neighbors (KNN) search, instantly retrieving the most semantically relevant route legs based on cosine distance[29].

This integration is highly efficient in cross-platform mobile frameworks. If the team utilizes React Native or Flutter, tools like react-native-nitro-sqlite or the flutter_gemma_rag_sqlite package allow for the static linking of sqlite-vec directly into the application's runtime environment[25]. By storing auxiliary non-vector data—such as the raw text of the route leg, fare constraints, and verification scores—in standard metadata columns adjacent to the vec0 table, the application eliminates the need for expensive cross-table SQL JOIN operations, ensuring lightning-fast retrieval times on constrained mobile CPUs[6].

Structuring Tacit Knowledge and Semantic Matching

The data schema outlined in the proposal is highly pragmatic, capturing the specific mode of transport, signboard identifiers, localized boarding landmarks, and dynamic fare ranges[1]. The critical technical hurdle is the quality of the vector embedding model.

Standard transit routing relies on exact geographic coordinate matching. Daanan relies on semantic matching. The chosen embedding model must be sufficiently trained on Philippine geographic context to understand that "SM Masinag" and "Masinag mall Antipolo" are semantically adjacent in vector space, despite massive lexical differences, while recognizing that "SM Megamall" represents a completely different geographic entity[1]. The team must ensure

the chosen embedding model is small enough (typically sub-100MB) to load into mobile RAM instantaneously without displacing the primary generative LLM[26].

Geospatial Routing and the Memory Crisis

The most perilous architectural decision in the Daanan proposal is the planned integration of GraphHopper or Valhalla utilizing OpenStreetMap (OSM) data to generate walking routes between transit legs entirely offline[1]. While GraphHopper is an industry-standard open-source routing engine, its deployment on mobile platforms introduces severe technical hazards that could easily induce system failure during the hackathon demonstration.

The GraphHopper Memory Bottleneck

GraphHopper relies on complex graph traversal algorithms, commonly accelerated by pre-processing techniques such as Contraction Hierarchies (CH) or Landmarks (LM)[31]. Processing raw OSM .pbf data into a routable base graph is computationally massive; parsing a planetary file requires upwards of 60GB of RAM and hours of CPU time, and even a constrained regional map demands significant memory to instantiate[31].

Historically, developers attempting to initialize a localized GraphHopper instance on an Android device frequently encounter catastrophic java.lang.OutOfMemoryError exceptions[21]. The native object allocation required to load the graph properties, edge values, and location indices rapidly breaches the Android Dalvik/ART virtual machine heap limits[21]. Compounding this risk, the official maintainers of GraphHopper deprecated direct offline mobile support after version 1.0, meaning modern forks require significant custom compilation and patching to function on contemporary Android environments, drawing vital engineering hours away from the core AI integration[33].

Mitigation Strategies for Offline Routing

To salvage this component of the proposal and prevent fatal memory crashes, the Daanan team must strictly adhere to the following deployment protocols:

1. **Off-Device Pre-Processing:** The mobile application must absolutely never parse raw OSM .pbf files on the device. The routing graphs must be pre-compiled on a high-memory desktop machine using the GraphHopper shell scripts (graphhopper.sh import) prior to deployment[21]. Only the heavily optimized binary outputs (e.g., nodes, edges, geometry, location_index) should be packaged into the mobile application asset folder[21].
2. **Memory-Mapped Files (MMAP):** To prevent the routing graph from consuming the entire Android heap, the application configuration must strictly enforce memory-mapped data access by setting graph.dataaccess.default_type: MMAP[31]. While MMAP results in marginally slower routing query times compared to loading the graph directly into RAM_STORE, it leverages the operating system's virtual memory management to stream data from flash storage, drastically reducing the active RAM footprint and preventing application crashes[32].
3. **Algorithmic Simplification:** The team must disable Contraction Hierarchies and Landmarks entirely for the mobile build[32]. While CH and LM vastly accelerate complex, long-distance driving queries by pre-calculating optimal paths, they bloat the required

graph size by gigabytes[32]. Daanan's routing engine is intended exclusively for short, localized pedestrian transfers between informal jeepney terminals and landmarks[1]. For walking distances under a few kilometers, standard Dijkstra or A* algorithms operate with sufficient speed on mobile CPUs without the massive memory overhead of CH preparation[32].

Strategic Pivot: Bypassing GraphHopper

If integrating GraphHopper proves too volatile or time-consuming during the 24-hour hackathon, the team must be prepared to pivot. A highly effective, low-risk alternative is to pre-calculate the walking distances and directional heuristics between known major transit nodes on the server side prior to the event. This data can be stored as static metadata within the sqlite-vec database. When a route is retrieved, the LLM can simply read the static distance data rather than relying on a complex, real-time spatial traversal engine, thereby entirely eliminating the risk of a geospatial-induced OOM crash.

Strategic Execution and Data Acquisition

The technical infrastructure outlined above is functionally useless without high-quality, localized transit data. The proposal accurately identifies that formal GTFS feeds are fundamentally unsuited for capturing the fluid, colloquial nature of Philippine jeepneys, necessitating the harvesting of informal community knowledge[1].

The Regulatory and Licensing Landscape

The proposal appropriately highlights the profound legal and operational risks of programmatically scraping community forums, explicitly citing Reddit's terms of service which strictly prohibit automated scraping and the commercial or AI-driven utilization of its user-generated content without explicit enterprise licensing agreements[1]. Attempting a bulk ingestion of r/HowToGetTherePH for a commercial launch would invite immediate legal scrutiny and potential API bans. Furthermore, freedom of information requests to government entities like the LTFRB yield data on public transportation routes that are often delayed, structurally rigid, and entirely disconnected from the colloquial reality of how commuters actually communicate[39].

However, for a hackathon prototype developed strictly under the principles of fair use and non-commercial academic demonstration, utilizing a highly constrained, hand-curated subset of forum data as a "seed" database is a standard and acceptable practice[1]. Moving forward, the proposed strategy of implementing an opt-in, one-tap contribution system ("Did this route work?") is vital to the project's longevity[1]. This shifts the platform from a legally precarious scraping engine to a self-sustaining, community-verified network, relying heavily on algorithmic freshness scoring to automatically deprecate outdated fares and altered routes over time[1].

Presentation and Demo Day Strategy

At the conclusion of the AppBuilders hackathon, the final product must be presented to a panel of judges in an aggressive 8-minute window (comprising 5 minutes for the pitch and live demonstration, followed by 3 minutes of Q&A)[2]. The proposal's strict adherence to a 99-second, 7-slide format derived from highly refined consulting methodologies and Y Combinator seed

deck templates is an exceptionally strong strategic advantage[1].

Constructing the Pitch Narrative

The presentation must immediately establish the unique insight derived from the "5 Whys" root-cause analysis: informal transit fails on existing routing applications not because mapping technology is flawed, but because the underlying data is tacit, unstructured, and inherently inaccessible when mobile connectivity fails[1]. The team must aggressively compress the YC pitch template: merging traction into validation metrics derived from their pre-hackathon interviews, consolidating the business model and market size into a single "Opportunity" slide, and stripping out all unnecessary technical jargon in favor of clear, resonant storytelling[1].

Pitch Slide	Duration	Core Strategic Objective
1. Hook	12s	Establish personal, relatable friction regarding informal commute confusion.
2. Problem at Scale	15s	Quantify the pain: 1,767 unconsolidated routes, hours lost, dead zones.
3. Solution	10s	Introduce Daanan: Offline Taglish AI powered by unstructured community data.
4. Demo Video / Live	30s	The core proof of concept. Must visually demonstrate offline capability.
5. How It Works	12s	Demystify the architecture: SLMs, vector DBs, no cloud dependence.
6. Opportunity	12s	Highlight market size and the failure of traditional GTFS feeds.
7. Close & Ask	8s	Summarize root cause and team capability.

Adapted from the proposed 99-second pitch format and AppBuilders Demo Day parameters[1].

Executing the Live Demonstration

The live demonstration is paramount, heavily influencing the "Product & Demo Quality" (15%) and "Local AI Implementation" (25%) scoring criteria[2]. The team must conduct a live, on-stage query with the presentation device placed explicitly in Airplane Mode[2]. This theatrical demonstration of true offline capability visually proves that the system does not depend on cloud APIs, satisfying the core mandate of the hackathon[2].

When disclosing technologies as required by the submission checklist, the team must be completely transparent regarding their use of edge models (Qwen 2.5), local frameworks (llama.cpp or LiteRT-LM), native databases (sqlite-vec), and any pre-compiled OSM graphs, addressing the specific hardware used for the demonstration[2]. Given the inherent latency of mobile CPU inference (which may hover around 5 to 15 tokens per second), the user interface must utilize streaming token generation, rendering the text onto the screen character-by-character as it is generated by the LLM[40]. This UI pattern masks processing latency and ensures the judges perceive the application as highly responsive, rather than waiting for a delayed bulk-text payload.

Final Conclusions and Recommendations

The Daanan proposal represents an exceptionally ambitious, highly relevant, and technically complex undertaking that aligns perfectly with the foundational spirit of the AppBuilders 2026 "Local AI" mandate[1]. By focusing on a profound localized problem—the digitization of informal transit networks in low-connectivity, developing-economy environments—the project demonstrates a deep, intuitive understanding of product-market fit[1].

However, the technical risks are immense. The convergence of large language models, vector databases, and geospatial routing engines on constrained mobile hardware threatens catastrophic memory failures that could easily destroy the final presentation. To ensure viability, the architecture must be strictly bounded and ruthlessly optimized for the hackathon environment.

The development team should immediately enforce the following protocols: abandon all attempts at mobile GPU acceleration in favor of stable CPU matrix instructions; select the smallest viable quantized Qwen 2.5 model to preserve system memory; pre-compile all SQLite vectors and GraphHopper routing nodes on desktop hardware prior to mobile deployment; and ensure the application strictly utilizes memory-mapped file access. By isolating these architectural vulnerabilities and focusing exclusively on demonstrating a flawless, airplane-mode query interface, Daanan possesses the requisite innovation, utility, and technical depth to be a dominant contender at the AppBuilders Demo Day.

Works cited

1. daanan-proposal.md
2. AppBuildersPH_Hackathon_2026_Participant_Briefing.md

3. Metro Manila/Jeepney and UV Express routes - OpenStreetMap Wiki, https://wiki.openstreetmap.org/wiki/Metro_Manila/Jeepney_and_UV_Express_routes
4. 1767 jeepney routes nationwide unconsolidated —LTFRB, https://www.gmanetwork.com/news/topstories/nation/894785/1-767-jeepney-routes-nationwide-unconsolidated-ltfrb/story/
5. sakayph/gtfs: GTFS feed for Metro Manila - GitHub, https://github.com/sakayph/gtfs
6. GitHub - asg017/sqlite-vec: A vector search SQLite extension that, https://github.com/asg017/sqlite-vec
7. Optimizing LLMs Using Quantization For Mobile Execution - arXiv, https://arxiv.org/html/2512.06490v1
8. Qwen 2.5 3B vs Llama 3.2 3B vs Phi-3.5 Mini vs Gemma 2 2B, https://www.generalcompute.com/blog/small-models-showdown-qwen-2-5-3b-llama-3-2-3b-phi-3-5-mini-gemma-2-2b
9. Introducing Qwen1.5 | Qwen, https://qwenlm.github.io/blog/qwen1.5/
10. What Is Qwen 3.5? Alibaba's Open-Weight Model That Runs on, https://www.mindstudio.ai/blog/what-is-qwen-3-5-alibaba-open-weight-model
11. qualcomm/Llama-v3.2-3B-Instruct - Hugging Face, https://huggingface.co/qualcomm/Llama-v3.2-3B-Instruct
12. Qwen 2.5 vs Llama 3.2 vs DeepSeek R1 - Premai.io, https://www.premai.io/blog/qwen-2-5-vs-llama-3-2-vs-deepseek-r1-enterprise-model-comparison-2026/
13. Small LLM Performance Benchmark - Research Report - AscentCore, https://ascentcore.com/2026/04/01/small-llm-performance-benchmark/
14. Qwen 2.5 on Phone: added 1.5B and 3B quantized versions ... - Reddit, https://www.reddit.com/r/LocalLLaMA/comments/1fkogmk/qwen_25_on_phone_added_15b_and_3b_quantized/
15. llama.cpp - Qwen, https://qwen.readthedocs.io/en/v1.5/run_locally/llama.cpp.html
16. For those building llama.cpp for Android (Snapdragon/Adreno only)., https://www.reddit.com/r/LocalLLaMA/comments/1o7p34f/for_those_building_llamacpp_for_android/
17. Benchmarking llama.cpp Backends on Intel Panther Lake: Vulkan vs, https://grigio.org/benchmarking-llama-cpp-backends-on-intel-panther-lake-vulkan-vs-sycl-vs-openvino-vs-cpu/
18. Performance of llama.cpp on Snapdragon X Elite/Plus #8273 - GitHub, https://github.com/ggml-org/llama.cpp/discussions/8273
19. llama.cpp memory and GGUF - Tokenminning, https://tokenminning.ai/self-hosting/llama-cpp/memory
20. Performance Study on COTS Mobile Devices - arXiv, https://arxiv.org/html/2410.03613v2
21. Graphhopper in Android app - gives OOM - Mobile, https://discuss.graphhopper.com/t/graphhopper-in-android-app-gives-oom/7831
22. LiteRT-LM Overview | Google AI Edge, https://developers.google.com/edge/litert-lm/overview

23. LiteRT Community (FKA TFLite) - Hugging Face, https://huggingface.co/litert-community
24. GitHub - google-ai-edge/LiteRT-LM, https://github.com/google-ai-edge/LiteRT-LM
25. flutter_gemma — On-device LLMs for Flutter, https://fluttergemma.dev/
26. flutter_gemma 1.11.2 | Flutter package - Pub.dev, https://pub.dev/packages/flutter_gemma/versions/1.11.2
27. sqlite-vec on Android and iOS devices - Alex Garcia, https://alexgarcia.xyz/sqlite-vec/android-ios.html
28. Introducing sqlite-vec v0.1.0: a vector search SQLite extension that, https://www.reddit.com/r/LocalLLaMA/comments/1ehlazq/introducing_sqlitevec_v010_a_vector_search_sqlite/
29. Fitting RAG in Your Pocket: Local Retrieval in React Native - Margelo, https://margelo.com/blog/fitting-RAG-in-your-pocket
30. Tracking issue: Android Support #68 - asg017/sqlite-vec - GitHub, https://github.com/asg017/sqlite-vec/issues/68
31. graphhopper/docs/core/deploy.md at master - GitHub, https://github.com/graphhopper/graphhopper/blob/master/docs/core/deploy.md
32. Host Your Own Worldwide Route Calculator With GraphHopper, https://www.graphhopper.com/blog/2022/06/27/host-your-own-worldwide-route-calculator-with-graphhopper/
33. GraphHopper Routing Engine 2.0 released, https://discuss.graphhopper.com/t/graphhopper-routing-engine-2-0-released/6016
34. Offlne routing on android - Mobile - GraphHopper Forum, https://discuss.graphhopper.com/t/offlne-routing-on-android/9176
35. Future of 'Offline' Routing · Issue #1940 · graphhopper ... - GitHub, https://github.com/graphhopper/graphhopper/issues/1940
36. Graphhopper, osmdroid for osmdroid offline routing - Stack Overflow, https://stackoverflow.com/questions/23021620/graphhopper-osmdroid-for-osmdroid-offline-routing
37. How much disk space do i need to build planet routing service, https://discuss.graphhopper.com/t/how-much-disk-space-do-i-need-to-build-planet-routing-service/1221
38. Philippines GTFS Schedule Feed - Mobility Database, https://mobilitydatabase.org/feeds/gtfs/mdb-1269
39. Public Transportation Routes within Metro/Mega Manila - FOI, https://www.foi.gov.ph/agencies/ltfrb/public-transportation-routes-within-metromega-manila/
40. llama.cpp: A CPU-First Framework for Running LLaMA Models on, https://www.sandgarden.com/learn/llama-cpp